Project Aya Technical Reference: Multilingual Distillation, Hybrid Mamba-MoE Architectures, and Hardware-Software Co-Design for Global Edge Deployment

The current landscape of artificial intelligence is characterized by a significant structural disparity between model capabilities and the global user base. While more than seventy percent of internet users are non-English speakers, the vast majority of efficient, high-performance language models remain English-centric in both training data and architectural optimization.[1] This gap is not merely a linguistic preference but a consequence of the computational costs associated with the Transformer architecture. The standard self-attention mechanism scales with quadratic time and memory complexity, denoted as $O(n^2)$, which creates a "computational exclusion" for users on consumer hardware and edge devices.[1] Project Aya represents a concerted effort to address this disparity by developing Aetheris, a hybrid Mamba-Mixture-of-Experts (MoE) model distilled from the 3.35B parameter Tiny Aya teacher.[1] The objective is to preserve multilingual reasoning and structured tool use in a student model of 500–800M parameters that can operate efficiently at the edge.[1]

The Problem of English-Centric Architectures and the Attention Bottleneck

The dominance of Transformer-based models has led to impressive results in language understanding and generation, but the quadratic scaling of softmax attention creates inherent limitations for long-context applications and low-resource environments.[2] As sequence lengths increase, the memory required for the Key-Value (KV) cache grows linearly, while the computation required for the attention matrix grows quadratically.[2] For multilingual models, this burden is compounded by variations in tokenization efficiency; many non-Latin scripts require more tokens to represent the same semantic information as English, effectively penalizing non-English speakers with higher latency and memory requirements.[1]

Metric	Transformer Baseline	Project Aya Objective (Aetheris)
Complexity	$O(n^2)$	$O(n)$ Selective State Space
Memory Constraint	Quadratic KV Cache growth	Constant hidden state size
Language Bias	English-centric optimization	70+ Languages / Equitable Compression

Deployment	Server-grade GPUs (H100/A100)	Edge AI / Consumer Hardware
Openness	Proprietary or Dense Weights	Open Science / Open Weights

The core research question driving Project Aya is whether these transformer-based capabilities can be distilled into Mamba architectures without sacrificing cross-lingual reasoning or structured performance.[1] This requires a fundamental shift toward subquadratic architectures like state-space models (SSMs), which provide linear inference complexity and a constant memory footprint.[2]

Architectural Paradigm: Aetheris and the Hybrid Mamba-MoE Framework

Aetheris utilizes a novel hybrid architecture that combines the strengths of selective state spaces with the efficiency of sparse Mixture-of-Experts.[1] The student model is designed to reach a parameter count between 500M and 800M, achieving a significant compression ratio compared to the Tiny Aya teacher.[1]

Selective State Spaces (SSM)

Mamba, as a structured state-space sequence model, introduces a selection mechanism that allows the model to filter out irrelevant information while retaining necessary data indefinitely by parameterizing the SSM based on the input.[3] Unlike traditional SSMs, which use fixed parameters for sequence mixing, Mamba's input-dependent parameters allow it to emulate the "Gather-and-Aggregate" (G&A) functionality of attention heads while maintaining the $O(n)$ scaling properties of a recurrent neural network.[2]

Sparse Mixture-of-Experts (MoE)

The integration of sparse MoE allows Aetheris to scale its total parameter count while keeping the active parameters during inference low.[1] The architecture employs four expert Feed-Forward Networks (FFNs) with top-1 routing and load balancing, ensuring that only one expert fires per token.[1] This design choice addresses the high computational cost of dense matrix operations in standard Transformers.[2]

Component	Designator	Specification
SSM Blocks	ψ	$O(n)$ selective scan with constant memory
Sparse MoE	Σ	4 expert FFNs; top-1 routing; load balancing
Hybrid Layers	Δ	SSM blocks on odd layers, MoE on even layers
Total Layers	L	24 layers with weight-tied embeddings

Precision	P	Gradient checkpointing enabled

The hybrid design places SSM blocks on odd layers and MoE blocks on even layers across 24 total layers.[1] This interleaving strategy is supported by research into models like Jamba and Zamba, which suggest that sparse attention or shared attention layers can improve the in-context learning (ICL) capabilities of pure SSMs.[6]

Theoretical Foundations of Cross-Architecture Distillation

The transfer of knowledge from a Transformer teacher to a Mamba student is handled through a multistage pipeline, specifically the MambaInLlama and MOHAWK frameworks.[9] These frameworks recognize that while Transformers and SSMs appear different, they can both be viewed as applying mixing matrices over token sequences.[5]

The MOHAWK Framework

The MOHAWK method (Matrix Orientation, Hidden-State Alignment, and Weight-Transfer Knowledge Distillation) targets different levels of granularity during the distillation process.[5]

1. **Matrix Orientation (Stage 1):** This phase aligns the sequence transformation matrices of the student and teacher.[5] By adjusting the student's matrix mixer to be structurally analogous to the teacher's, the distance between the layer outputs is minimized.[9]
2. **Hidden-State Alignment (Stage 2):** This stage aligns the hidden-state representations of each block.[5] It has been observed that the model's ability to recover teacher knowledge is strongly correlated with the alignment of these internal activations.[9]
3. **End-to-End Distillation (Stage 3):** The final stage fine-tunes the entire student model to match the teacher's predictions.[5]

Weight Initialization and Linearized Attention

A critical insight for the Project Aya pipeline is the mathematical connection between linearized attention and linear RNNs.[10] If the softmax nonlinearity is removed from attention, the resulting formula can be expressed in a recurrent form.[12] This allows the student Mamba model to be initialized directly from the teacher's attention weights.[10]
In this scheme, the standard Query (Q), Key (K), and Value (V) weights are mapped to the SSM parameters A, B, C, and input x_t [12]:

- x_t is initialized from the Value projection (W_V).[12]
- B_t is initialized from the Key projection (W_K).[12]
- C_t is initialized from the Query projection (W_Q).[12]
- A_t is initialized to represent the positional indicators/masks of the teacher.[12]

Projection	**Transformer Weight**	**Mamba Parameter**

Value	W_V	x_t (Input)
Key	W_K	B_t (Input Gate)
Query	W_Q	C_t (Output Gate)
Mask/Pos	$m_{t-1,t}$	A_t (State Transition)

This structured initialization accelerates convergence and ensures that the student preserves the teacher's "knowledge," which is hypothesized to be stored largely in the MLP layers and the pre-head norms.[9] During the initial distillation phases, these MLP layers are often kept frozen to act as a stable knowledge base while the Mamba layers are trained to master sequence mixing.[11]

Multilingual Equity: TRepLiNa and Representational Alignment

The primary challenge in multilingual distillation is preventing "representation drift" in low-resource languages (LRLs). As a model is compressed, the performance on LRLs often degrades faster than on high-resource languages (HRLs), a phenomenon Project Aya seeks to mitigate through the TRepLiNa framework.[1]

Centered Kernel Alignment (CKA) and REPINA

TRepLiNa combines Centered Kernel Alignment (CKA) with Representation Projection Invariance (REPINA).[14] CKA is a similarity metric used to encourage the internal representations of different languages to align within the model's shared space.[14] By applying a CKA loss to the hidden representations of parallel sentences (e.g., Santali and English), the model learns to map the LRL input into the HRL reasoning subspace.[14]

However, CKA alone can cause the HRL features to drift, potentially damaging the model's strongest capabilities.[4] REPINA acts as a stabilizer by anchoring the HRL states to a reference pass of the pretrained model (with adapters disabled), constraining parameter updates to remain close to the teacher's original features.[14]

The joint loss function is defined as:

$$L = L_{MT} + \lambda L_{CKA} + \mu L_{REPINA}$$

where L_{MT} is the standard machine translation loss, λ is the weight for CKA alignment, and μ is the weight for REPINA anchoring.[15]

Optimal Alignment Layers

Systematic studies of Aya-23 8B using TRepLiNa indicate that alignment is most effective in the intermediate layers, specifically layers 10 through 15.[4] Research suggests that multilingual models transform language-specific inputs into more language-agnostic representations in these middle layers.[4] While CKA-only alignment peaks at layer 10, the combination with REPINA favors layer 15 for data-scarce settings.[4]

Feature	**CKA-Only**	**TRepLiNa (CKA+REPINA)**
LRL Trajectory	Drifts toward HRL	Drifts toward HRL
HRL Trajectory	Drifts toward LRL	Anchored to Reference
Optimal Layer	10	15
Stability	Low (Representation Drift)	High (Guided Alignment)
Cost	Low (Lightweight Loss)	Low (Lightweight Loss)

Experiments on Bhili, Gondi, Mundari, and Santali demonstrate that this targeted alignment can achieve significant relative improvements in translation quality, such as the 2.27x improvement observed for Santali-to-English translation.[4]

Hardware-Software Co-Design: Optimization for Blackwell

For edge deployment to be viable, the model must be optimized for the specific hardware bottlenecks of modern GPUs. The transition from NVIDIA Hopper to Blackwell architectures has shifted the primary bottleneck from matrix multiplication to the Special Function Units (SFUs) used for softmax and shared memory bandwidth.[18]

FlashAttention-4 and the SFU Bottleneck

On Blackwell GPUs, tensor core throughput has doubled (reaching up to 2.25 PFLOPs for BF16), but SFU throughput for operations like exponential (exp) has remained unchanged.[20] This makes the softmax operation in attention layers a critical bottleneck. FlashAttention-4 (FA4) addresses this through several techniques [18]:

- **Software-Emulated Exponential:** FA4 distributes exponential computations across hardware MUFUs and software emulation on underutilized Floating Point Multiply-Add (FMA) units.[20]
- **Ping-Pong Scheduling:** The kernel calculates two query tiles per Streaming Multiprocessor (SM) in a ping-pong schedule to maximize overlap between matrix-multiply-accumulate (MMA) operations and softmax.[20]
- **Tensor Memory (TMEM):** FA4 leverages the 256 KB of Tensor Memory per SM on Blackwell GPUs to store intermediate results, reducing register pressure and shared memory traffic.[18]

Hardware Feature	**Hopper (H100)**	**Blackwell (B200)**
BF16 Tensor Cores	1 PFLOP	2.25 PFLOPs
TMEM (per SM)	N/A	256 KB
Shared Memory BW	1x	1x (No Change)
SFU (exp) Count	1x	1x (No Change)
MMA Tile Size	64x128	128x128 (Direct to TMEM)

CuTe-DSL and Kernel Development

FlashAttention-4 is implemented entirely in CuTe-DSL, a Python-embedded domain-specific language that lowers to PTX and then to assembly.[18] This approach achieves 20–30x faster compile times compared to traditional C++ template metaprogramming while maintaining full expressivity and PTX-level control.[18] This rapid iteration is crucial for the Aetheris team as they prototype hybrid Mamba-MoE kernels that must balance sequence mixing with expert routing on the same hardware clusters.

Linguistic Typology and Compression Resilience

The evaluation of Aetheris spans ten representative languages across five distinct typological families to measure the equity of the distillation process.[1] Different typologies—analytic, agglutinative, fusional, and polysynthetic—interact differently with the subword tokenizers and state-space selection mechanisms used in LLMs.[1]

Typological Categories in Evaluation

- **Analytic (e.g., English, Mandarin, Indonesian):** These languages rely on word order and auxiliary words rather than morphological changes to express grammatical relationships.[1]
- **Agglutinative (e.g., Swahili, Turkish, Japanese, Telugu):** These languages pack grammatical information into words by adding distinct, unchangeable suffixes.[1]
- **Fusional (e.g., Spanish, Hindi, Arabic):** These languages use suffixes that may merge several grammatical features into a single morpheme, making the boundaries between features less clear.[1]

Language	Family	Script	Typology
English	Indo-European	Latin	Analytic
Spanish	Indo-European	Latin	Fusional
Hindi	Indo-European	Devanagari	Fusional
Mandarin	Sino-Tibetan	CJK	Analytic/Isolating
Arabic	Afroasiatic	Arabic	Fusional/Semitic
Swahili	Niger-Congo	Latin	Agglutinative
Turkish	Turkic	Latin	Agglutinative
Japanese	Japonic	CJK+Kana	Agglutinative
Indonesian	Austronesian	Latin	Analytic
Telugu	Dravidian	Telugu	Agglutinative

The **Degradation Equity Score** is a novel metric introduced by the Project Aya team to measure the variance of accuracy drop across these language families after distillation.[1] A lower score indicates that compression is "equitable," meaning it does not pick "winners and losers" among linguistic typologies.[1] This is critical for ensuring that edge deployment does not inadvertently favor users of analytic languages over those using more morphologically complex ones.

Comprehensive Evaluation Ecosystem

The success of Aetheris is measured through a suite of benchmarks that test reasoning, arithmetic, and structured interaction in a multilingual context.

mGSM: Multilingual Math

The Multilingual Grade School Math (mGSM) benchmark evaluates arithmetic reasoning across 10 languages.[1] It consists of 250 problems manually translated to ensure high fidelity and quality control.[24] Research indicates that while models are generally strongest in English, "Native-CoT" (performing the chain-of-thought in the target language) is essential for accurate reasoning in underrepresented languages like Bengali or Telugu.[24]

BenchMAX 2026

BenchMAX is a high-quality, parallel multilingual benchmark comprising 10 tasks across 17 languages.[28] It assesses instruction following, code generation, long-context modeling, and translation.[28]

BenchMAX Task	Capability Evaluated	Dataset Link
Multiple Functions	Tool Use / Multilingual Function Calling	BenchMAX_Multiple_Functions
Math & Science	Reasoning / Logic	BenchMAX_Math / Science
Function Completion	Code Generation	BenchMAX_Function_Completion
Rule/Model-Based	Instruction Following	BenchMAX_Rule/Model-based
Question Answering	Long Context Modelling	BenchMAX_Question_Answering

The Tool Use benchmark is particularly significant, as it evaluates structured JSON generation and function selection in 10+ languages—a capability that remains nearly unstudied in multilingual settings.[1] The evaluation uses modified NexusRaven code to ensure models can handle model-specific output formats, such as those required by Llama 3.[28]

Advanced Inference and Scaling Strategies

Beyond the architectural design of Aetheris, the Project Aya research roadmap incorporates advanced inference techniques to enhance the performance of the compressed student model.

Fusion-of-N (FusionN)

Traditional Best-of-N (BoN) approaches select a single winning generation from a pool of samples, which is a zero-sum process.[29] Project Aya investigates Fusion-of-N (FusioN), which uses a strong LLM judge (the fusor) to synthesize the informative elements of multiple candidates into a single final answer.[29] This collaborative setup integrated across multiple generation paths allows the student model to integrate diverse strengths that might be inaccessible through selection alone.[29]

ToolQP and Socratic CoT

To address the limitations of small models in complex agentic tasks, the project explores "Socratic CoT" and "TOOLQP".[30] In Socratic CoT, one model acts as a "decomposer" to break a query into smaller parts, while a "solver" addresses each part and assembles the final output.[31] TOOLQP similarly bridges semantic gaps by decomposing instructions into sub-tasks and dynamically generating queries to interact with a massive tool library, circumventing the need to fit the entire tool context into the LLM's window.[30]

Project Roadmap and Implementation Tracker

Project Aya operates on a rigorous schedule with the goal of releasing open-science / open-weight models by Q3 2026.[1]

Milestone Overview

Phase	Name	Key Deliverables	Target
01	Baseline	CPU tokens/sec; Tokenizer validation; Performance baselines	Q1 2026
02	Distillation	RunPod A100 training; Layer alignment; KL Distillation; SFT	Q2 2026
03	Evaluation	Equity analysis; Benchmarking (mGSM, XCOPA, Tooling); Paper draft	Q3 2026

Team Task Assignment

The following tracker manages the progress of individual components across the project phases.[1]

Phase	Task	Status	Owner
Phase 01	Environment setup & dependency validation	Not Started	TBD
Phase 01	Establish CPU baseline (tokens/sec) in 5 languages	Not Started	TBD

Phase 02	Layer alignment: attention $\to$ Mamba SSM blocks	Not Started	TBD
Phase 02	KL distillation training on RunPod A100	Not Started	TBD
Phase 02	Supervised fine-tuning: multilingual + tool calling	Not Started	TBD
Phase 03	Run mGSM benchmark (10 languages)	Not Started	TBD
Phase 03	Compute Degradation Equity Score across families	Not Started	TBD
Phase 03	Open-weight model release	Not Started	TBD

The project utilizes a status key where **Not Started** indicates work has not yet begun, **In Progress** denotes active development, and **Complete** signifies verified results.[1]

Collaborative Synthesis and Future Outlook

Project Aya represents a holistic approach to the problem of multilingual AI efficiency. By combining mathematical cross-architecture distillation (MambaInLlama/MOHAWK) with innovative representational alignment (TRepLiNa), hardware-aware co-design (FA4), and diverse typological evaluation, the project aims to define a new standard for linguistic inclusion.[1] The focus on edge-ready deployment via a hybrid Mamba-MoE architecture ensures that the benefits of large-scale multilingual modeling are no longer restricted to high-resource environments.

The development of the Aetheris model is supported by an international team of researchers from institutions such as Wayy Research, IIT Kharagpur, Stanford, IISc Bangalore, and Kensho/S&P Global.[1] This collaborative effort emphasizes the importance of open weights and open science in addressing the systemic gaps in language technology. As the project moves into Phase 02, the technical focus will remain on the high-efficiency transfer of information from the Tiny Aya Transformer to the Mamba architecture, a challenge that remains at the forefront of contemporary AI research.[1] The ultimate delivery of a sub-1B parameter model capable of reasoning across 70+ languages will provide a critical proof-of-concept for the future of democratic AI access.[1]

Works cited

1. Project_Aya_Team_Doc.docx
2. The Mamba in the Llama: Distilling and Accelerating Hybrid Models - ResearchGate, accessed March 6, 2026, https://www.researchgate.net/publication/397202800_The_Mamba_in_the_Llama_Distilling_and_Accelerating_Hybrid_Models
3. A Survey of Mamba - arXiv, accessed March 6, 2026,

https://arxiv.org/html/2408.01129v5
4. TRepLiNa: Layer-wise CKA+REPINA Alignment Improves Low-Resource Machine Translation in Aya-23 8B - arXiv, accessed March 6, 2026, https://arxiv.org/html/2510.06249v5
5. Transformers to SSMs: Distilling Quadratic Knowledge to Subquadratic Models - NIPS, accessed March 6, 2026, https://proceedings.neurips.cc/paper_files/paper/2024/file/3848fef259495bfd04d60cdc5c1b4db7-Paper-Conference.pdf
6. LongLLaVA: Scaling Multi-modal LLMs to 1000 Images Efficiently via a Hybrid Architecture, accessed March 6, 2026, https://openreview.net/forum?id=wqA7QmpUwa
7. Zamba - Hugging Face, accessed March 6, 2026, https://huggingface.co/docs/transformers/model_doc/zamba
8. [AINews] Mixtral 8x22B Instruct sparks efficiency memes - Buttondown, accessed March 6, 2026, https://buttondown.com/ainews/archive/ainews-mixtral-8x22b-instruct-defines-frontier/
9. Cross-Architecture Distillation Part I - The MOHAWK Framework ..., accessed March 6, 2026, https://goombalab.github.io/blog/2024/distillation-part1-mohawk/
10. MaTVLM: Hybrid Mamba-Transformer for Efficient Vision-Language Modeling - arXiv, accessed March 6, 2026, https://arxiv.org/html/2503.13440v1
11. The Mamba in the Llama: Distilling and Accelerating Hybrid Models | Read Paper on Bytez, accessed March 6, 2026, https://bytez.com/docs/neurips/93289/paper?_c=eyJ2IjoxLCJyZWxhdGVkIjpbImNvZGUiLCJyZWZlcmVuY2VzIiwiY29uZmVyZW5jZSJdfQ%3D%3D
12. The Mamba in the Llama: Distilling and ... - NIPS - NeurIPS, accessed March 6, 2026, https://proceedings.neurips.cc/paper_files/paper/2024/file/723933067ad315269b620bc0d2c05cba-Paper-Conference.pdf
13. AMD-HybridLM: Towards Extremely Efficient Hybrid Language Models — ROCm Blogs, accessed March 6, 2026, https://rocm.blogs.amd.com/artificial-intelligence/hybrid-models,-mla,/README.html
14. TRepLiNa: Layer-wise CKA+REPINA Alignment Improves Low-Resource Machine Translation in Aya-23 8B - arXiv, accessed March 6, 2026, https://arxiv.org/html/2510.06249v1
15. TRepLiNa: Layer-wise CKA+REPINA Alignment Improves Low-Resource Machine Translation in Aya-23 8B - arXiv.org, accessed March 6, 2026, https://arxiv.org/pdf/2510.06249
16. TRepLiNa: Layer-wise CKA+REPINA Alignment Improves Low-Resource Machine Translation in Aya-23 8B - ACL Anthology, accessed March 6, 2026, https://aclanthology.org/2025.mmloso-1.3.pdf
17. TRepLiNa: Layer-wise CKA+REPINA Alignment Improves ... - arXiv, accessed March 6, 2026, https://arxiv.org/abs/2510.06249
18. FlashAttention-4: Algorithm and Kernel Pipelining Co-Design for Asymmetric

Hardware Scaling - arXiv.org, accessed March 6, 2026, https://arxiv.org/html/2603.05451v1
19. FlexAttention + FlashAttention-4: Fast and Flexible - PyTorch, accessed March 6, 2026, https://pytorch.org/blog/flexattention-flashattention-4-fast-and-flexible/
20. FlashAttention-4: Algorithm and Kernel Pipelining Co-Design for ..., accessed March 6, 2026, https://www.together.ai/blog/flashattention-4
21. FlashAttention-4 Officially Released: Major Overhaul of Algorithm Pipeline with Matrix Multiplication-Level Speed - 36氪, accessed March 6, 2026, https://eu.36kr.com/en/p/3711195049046148
22. Proceedings of the 29th International Conference on Computational Linguistics - ACL Anthology, accessed March 6, 2026, https://aclanthology.org/volumes/2022.coling-1/
23. Bridging Linguistic Typology and Multilingual Machine Translation with Multi-View Language Representations | Request PDF - ResearchGate, accessed March 6, 2026, https://www.researchgate.net/publication/347235116_Bridging_Linguistic_Typology_and_Multilingual_Machine_Translation_with_Multi-View_Language_Representations
24. MGSM Benchmark: Multilingual Grade School Math - Emergent Mind, accessed March 6, 2026, https://www.emergentmind.com/topics/multilingual-grade-school-math-mgsm-benchmark
25. MGSM: Multilingual Grade School Math - GitHub Pages, accessed March 6, 2026, https://ukgovernmentbeis.github.io/inspect_evals/evals/mathematics/mgsm/
26. Pushing on Multilingual Reasoning Models with Language-Mixed Chain-of-Thought, accessed March 6, 2026, https://openreview.net/forum?id=ABc5y3741T
27. MGSM - Vals AI, accessed March 6, 2026, https://www.vals.ai/benchmarks/mgsm
28. CONE-MT/BenchMAX - GitHub, accessed March 6, 2026, https://github.com/CONE-MT/BenchMAX
29. Making, not Taking, the Best of N - arXiv, accessed March 6, 2026, https://arxiv.org/pdf/2510.00931
30. Beyond Single-Shot: Multi-step Tool Retrieval via Query Planning - arXiv.org, accessed March 6, 2026, https://arxiv.org/pdf/2601.07782
31. Distilling Reasoning Capabilities into Smaller Language Models | Request PDF, accessed March 6, 2026, https://www.researchgate.net/publication/372918028_Distilling_Reasoning_Capabilities_into_Smaller_Language_Models